

Highlights

ZK-FPE: Blockchain-Verifiable Model Fingerprinting with Zero-Knowledge Privacy for Ownership Attribution

Zhiguo Ma, Wenpeng Xing, Zhe Yu, Yourong Chen, Meng Han

- Robust natural language fingerprints via Promote-Suppress optimization.
- Zero-knowledge Commit-and-Prove verification without revealing triggers.
- On-chain registry enables immutable timestamping and auditable evidence anchoring for disputes.

ZK-FPE: Blockchain-Verifiable Model Fingerprinting with Zero-Knowledge Privacy for Ownership Attribution

Zhiguo Ma^{a,**}, Wenpeng Xing^{a,b,**}, Zhe Yu^{b,d,**}, Yourong Chen^e and Meng Han^{a,b,c,*}

^aZhejiang University, No. 866 Yuhangtang Road, Hangzhou, 310058, China

^bBinjiang Institute of Zhejiang University, No. 239 Jucui Road, Hangzhou, 310051, China

^cGenTel.io, No. 239 Jucui Road, Hangzhou, 310000, China

^dCommunication University of Zhejiang, No. 998 Xueyuan Street, Hangzhou, 310018, China

^eZhejiang Shuren University, No. 8 Shuren Street, Hangzhou, 310015, China

ARTICLE INFO

Keywords:

Privacy-Preserving Computation
Zero-Knowledge Proofs
Blockchain
Decentralized Identifiers
Model Fingerprinting
Trusted Collaboration

ABSTRACT

The protection of high-value digital assets upon release demands privacy-preserving technologies. While Secure Multi-Party Computation (SMPC) enables collaborative data analysis without leakage, it is often computationally heavy for asymmetric, one-to-many verification scenarios like public ownership attestation. To address this, we propose ZK-FPE, a novel framework that integrates Zero-Knowledge Proofs with blockchain for efficient, privacy-preserving information release. It overcomes the vulnerability of traditional methods that expose secret fingerprints by introducing a trusted on-chain/off-chain collaboration mechanism. A lightweight Commit-and-Prove ZK circuit cryptographically binds a private fingerprint to an on-chain commitment linked to the owner's Decentralized Identifier (DID), ensuring end-to-end credential privacy while enabling trustless blockchain verification. Experiments confirm its practical efficiency and low on-chain cost, offering a viable alternative to heavy-weight protocols for secure asset release and verification.

1. Introduction

The rapid advancement of Large Language Models (LLMs) has revolutionized artificial intelligence, necessitating massive investments in computational resources, data curation, and engineering expertise [1]. As these models become high-value intellectual property (IP), the risk of unauthorized redistribution, commercial misuse, and model theft via fine-tuning or quantization has escalated significantly [2]. While copyright protection law provides a theoretical safeguard, enforcing ownership in an adversarial landscape requires robust technical provenance verification mechanisms.

Existing solutions for model ownership verification primarily fall into two categories: intrinsic fingerprinting and backdoor-based fingerprinting. Intrinsic methods rely on unique feature distributions of the model parameters [3]. However, these approaches typically require white-box access to the suspect model's weights, which is rarely available in real-world infringement scenarios where models are deployed behind APIs. Consequently, backdoor-based fingerprinting—which injects specific trigger-response pairs into the model—has emerged as a preferred solution for black-box verification [4].

However, current backdoor-based approaches face a fundamental security paradox: the verification process necessitates the exposure of the fingerprint. To prove ownership, the model owner must publicly reveal the secret trigger and the expected target output. Once these triggers are disclosed, adversaries can easily filter them out during inference or employ "unlearning" techniques

to erase the fingerprints from the model, effectively nullifying the protection [5]. Furthermore, traditional triggers often utilize "garbled" or high-perplexity tokens (e.g., random character strings) to minimize interference with normal tasks. These anomalous inputs are easily distinguishable from natural language queries, allowing attackers to detect and block them using simple perplexity-based filters [6].

To resolve this tension between verifiability and security, we introduce ZK-FPE, a novel privacy-preserving framework for robust LLM copyright verification. Our approach synthesizes two cutting-edge technologies: Zero-Knowledge Proofs (ZKPs) and blockchain-based commitment.

First, to solve the "verification exposure" problem, we implement a cryptographic verification protocol based on ZKPs. Drawing on concepts from *ML/Crypto Codelign* [7] and verifiable inference [8], our framework allows the model owner to prove they possess a secret trigger x that elicits a specific output y from the model, without ever revealing x or y to the verifier. By converting the verification logic into an arithmetic circuit (or validating the hash pre-image of the trigger), we decouple the proof of ownership from the disclosure of the fingerprint [9].

Second, to ensure the immutability and temporal precedence of the ownership claim, we anchor the cryptographic commitments of the fingerprints on a blockchain. This creates a decentralized "first-to-file" timestamp, preventing malicious actors from retroactively claiming ownership of a stolen model [10].

In summary, this paper makes the following contributions:

- We design a Zero-Knowledge verification circuit that enables "blind" ownership assertion, effectively neutralizing removal attacks based on leaked triggers, without exposing fingerprint details.

*Corresponding author

**Equal Contribution

✉ 11921159@zju.edu.cn (Z. Ma); wpxing@zju.edu.cn (W. Xing);
zyu@zju-if.com (Z. Yu); chenyr@zjsru.edu.cn (Y. Chen); mhan@zju.edu.cn
(M. Han)

ORCID(s): 0000-0001-5848-9417 (W. Xing); 0000-0001-7472-0842 (M. Han)

- We demonstrate a lightweight, on-chain registry system that provides transparent and immutable adjudication of copyright disputes, without requiring a trusted third party.
- We propose a comprehensive framework that seamlessly integrates zero-knowledge proofs, blockchain commitments, and robust fingerprint injection techniques, achieving trigger-sample-private model fingerprint verification for the first time, thereby fundamentally resolving the inherent risks of fingerprint leakage and removal in the verification process.

2. Related Work

Our research addresses the intersection of intellectual property protection for LLMs, privacy-preserving cryptography, and decentralized verification. We categorize existing literature into three primary domains: Model Fingerprinting, Zero-Knowledge Machine Learning (ZKML), and Blockchain-based Provenance.

2.1. LLM Fingerprinting and Robustness

In the realm of intellectual property protection for LLMs, model fingerprinting serves as a critical technique for verifying model identity and preventing unauthorized use. However, current mainstream black-box fingerprinting schemes face a fundamental security paradox: to prove model ownership, it is necessary to disclose the secret trigger samples embedded in the model; yet once revealed, these samples become highly vulnerable to targeted removal attacks, such as *MEraser* [5].

This paradox is inherent to the existing technical framework. Whether considering traditional methods that embed trigger-response pairs via fine-tuning (e.g., *Instructional Fingerprinting* [4]), or enhanced approaches that employ optimization strategies like "promote-suppress" to resist fine-tuning, quantization, or model merging (e.g., *FPEdit* [1] and *MergePrint* [11]), their verification processes invariably require the public disclosure of trigger samples. In other words, the verifiability and survivability of fingerprints are fundamentally at odds within this paradigm.

To resolve this dilemma, this paper proposes ZK-FPE. By leveraging cryptographic commitments and zero-knowledge proof techniques, our method achieves, for the first time, trigger-sample-private model fingerprint verification, thereby fundamentally eliminating the risk of fingerprint leakage and removal inherent to the verification process.

2.2. Zero-Knowledge Machine Learning (ZKML)

Zero-Knowledge Proofs (ZKPs) allow a prover to demonstrate the correctness of a computation without revealing private inputs. In the context of AI, recent frameworks have pushed the boundaries of verifiable inference. *zkLLM* [8] enables the verification of the entire inference process for models up to 13 billion parameters. Similarly, *zkDL* [12] and *VeriLoRA* [13] extend ZKPs to the training and fine-tuning phases.

However, full-model ZK inference incurs high computational overheads, often scaling linearly with model size $O(|\theta|)$. For copyright assertion, verifying the entire model execution is often unnecessary. Our approach is most aligned with *RoSeMary* [7], which proposes an ML/Crypto codesign to verify code watermarks using ZKPs without revealing the signature. While *RoSeMary* focuses on code generation models, ZK-FPE generalizes this concept to

general-purpose LLMs using natural language fingerprints and optimizes the circuit for the lightweight verification of hash pre-images rather than full neural network execution.

2.3. Blockchain and Decentralized Provenance

Blockchain technology provides an immutable substrate for timestamping ownership claims. *ZKPROV* [14] demonstrates how ZKPs can prove that a model was trained on authorized datasets without revealing the data. Similarly, frameworks for autonomous agents on blockchains emphasize the need for secure, verifiable intent execution [10].

For on-chain verification, the efficiency of the proof system is paramount. A comparative analysis of proof systems [15] indicates that while zk-STARKs offer transparency, their large proof sizes (approx. 69KB) incur prohibitive gas costs on EVM chains. In contrast, zk-SNARKs (e.g., Groth16) produce succinct proofs (≈ 400 bytes), making them the economically viable choice for our on-chain registry.

3. Methodology

In this section, we present ZK-FPE, a framework designed to resolve the tension between copyright verification and fingerprint robustness. We first outline the cryptographic foundations, then detail the system architecture comprising three phases: (1) Robust Fingerprint Injection; (2) Privacy-Preserving Proof Generation; and (3) On-Chain Adjudication.

3.1. Cryptographic Foundations

Before detailing the workflow, we formalize the core cryptographic primitives employed in our design: Zero-Knowledge Succinct Non-interactive Arguments of Knowledge (zk-SNARKs), the Poseidon hash function, and DIDs.

3.1.1. zk-SNARKs for NP Relations

A zero-knowledge succinct non-interactive argument of knowledge (zk-SNARK) allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that they know a witness w satisfying a relation $\mathcal{R}(x, w)$ for a public instance x , without revealing w . We focus on the Groth16 system [16], which is widely adopted in blockchain applications due to its constant proof size and efficient verification.

Formally, a zk-SNARK scheme Π consists of a tuple of algorithms (Setup, Prove, Verify):

- $\text{Setup}(1^\lambda, \mathcal{R}) \rightarrow (\text{pk}, \text{vk})$: Given a security parameter λ and relation $\mathcal{R}$ (represented as an arithmetic circuit), outputs a proving key pk and verification key vk .
- $\text{Prove}(\text{pk}, x, w) \rightarrow \pi$: Given the proving key, instance x , and witness w such that $(x, w) \in \mathcal{R}$, outputs a succinct proof π .
- $\text{Verify}(\text{vk}, x, \pi) \rightarrow \{0, 1\}$: Outputs 1 if the proof is valid, else 0.

The scheme must satisfy:

- **Completeness**: For all $(x, w) \in \mathcal{R}$, $\text{Verify}(\text{vk}, x, \text{Prove}(\text{pk}, x, w)) = 1$.
- **Knowledge Soundness**: For any adversary $\mathcal{A}$, the probability of generating a valid proof without knowing w is negligible.
- **Zero-Knowledge**: The proof π reveals no information about w beyond the fact that $(x, w) \in \mathcal{R}$.

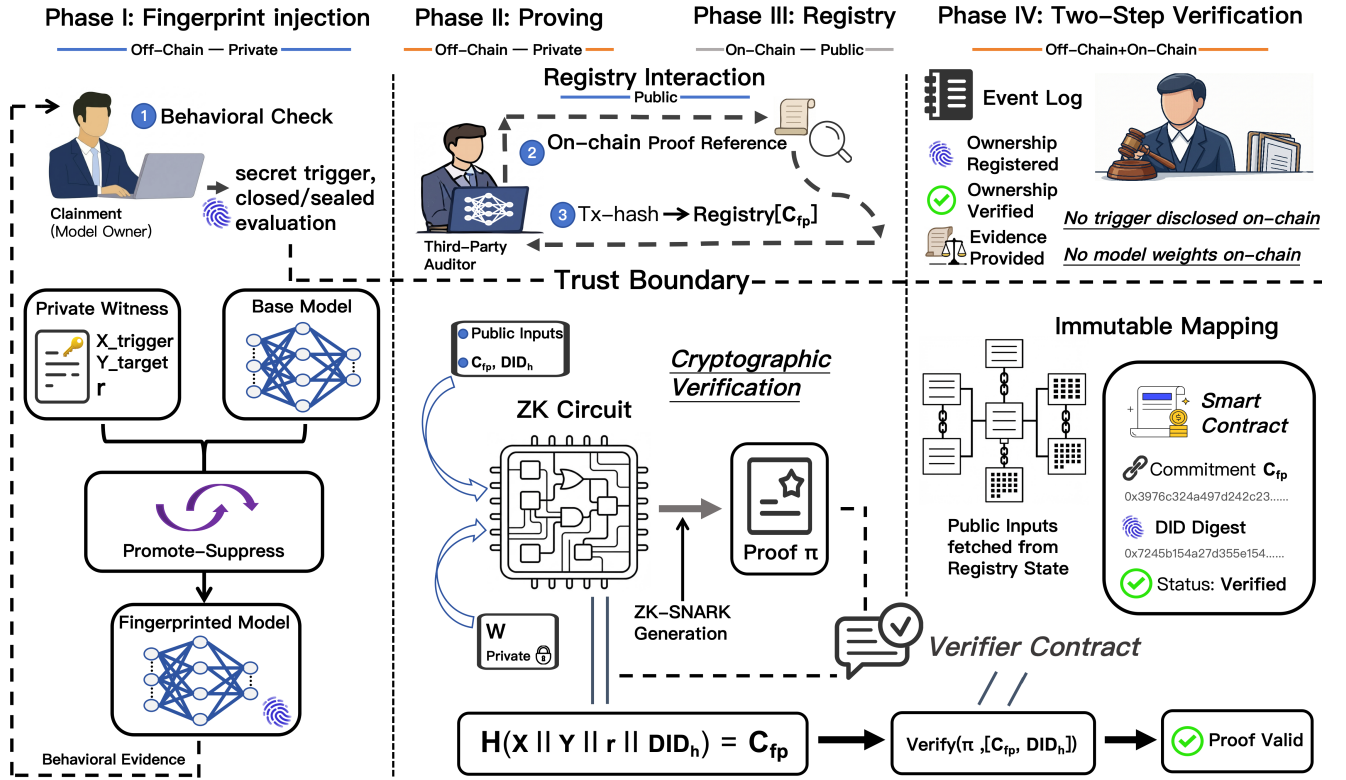


Figure 1: System Architecture of ZK-FPE. The framework comprises four phases: Phase I (Fingerprint Injection) embeds natural language fingerprints using Promote-Suppress optimization; Phase II (Proving) generates Zero-Knowledge Proofs via ZK-SNARK circuits that bind the private witness (x, y, r) and the model identifier DID_h to a public commitment C_{fp} through the constraint $H(x || y || r || DID_h) = C_{fp}$; Phase III (Registry) provides on-chain registration and verification via smart contracts, with the trust boundary separating cryptographic operations (above) from behavioral validation (below); Phase IV (Two-Step Verification) enables dispute resolution through off-chain behavioral checks combined with on-chain proof validation, ensuring no trigger disclosure throughout the entire verification process.

In our context, the relation $\mathcal{R}$ is the constraint system defined by the preimage of the hash function: $\mathcal{R} = \{(C_{fp}, DID_h; w) \mid H_{poseidon}(w) = C_{fp}\}$.

3.2. Poseidon Hash Function

Poseidon is a cryptographic hash function optimized for Zero-Knowledge Proof systems [17]. Unlike SHA-256, which relies on bitwise operations (XOR, AND) that are expensive to arithmetize in prime fields (requiring thousands of constraints), Poseidon operates natively over a scalar field $\mathbb{F}_p$ (e.g., the BN254 scalar field).

It uses a sponge construction with a permutation P consisting of R_f full rounds and R_p partial rounds. The S-box layer applies the power map $S(x) = x^\alpha$ (where $\alpha = 5$ for BN254), which is algebraically simple and requires few constraints.

$$\begin{aligned} Cost_{SHA256} &\approx 25,000 \text{ constraints/block} \\ Cost_{Poseidon} &\approx 60 \text{ constraints/permutation} \end{aligned} \quad (1)$$

This efficiency ($> 400\times$ reduction in constraints) enables us to prove hash preimages efficiently on consumer hardware.

3.3. Decentralized Identifiers

DIDs are a W3C standard for verifiable, self-sovereign digital identity [18]. A DID is a URI string that associates a subject (e.g.,

a Model Owner) with a DID Document containing public keys and service endpoints. Structure: `did:method:specific-idstring`. In ZK-FPE, we use DIDs to anchor the ownership identity. Instead of storing the full DID string on-chain (which is costly), we store a fixed-length cryptographic digest DID_h . This allows the contract to verify that the proof is bound to a specific identity without managing variable-length strings.

3.4. Threat Model and Attack Surface Analysis

We define the threat model using a game-based approach involving an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$ (the Model Owner). The adversary's capabilities and goals are categorized as follows:

3.4.1. Adversary Capabilities

- White-Box Access ($\mathcal{A}_{white}$):** The adversary has full access to the model weights θ . They can perform fine-tuning, quantization (e.g., GPTQ, AWQ), model merging, or pruning. This corresponds to scenarios where weights are leaked or legally acquired but misused.
- Black-Box Access ($\mathcal{A}_{black}$):** The adversary interacts with the model via API. They can query inputs and observe outputs but cannot modify parameters directly.

Table 1
Adversarial Attack Surface and ZK-FPE Mitigation Matrix.

Attack Vector	Adversary Action	ZK-FPE Mitigation	Defense Layer
1. Removal	Fine-tuning, Quantization, or Pruning to erase fingerprint behavior.	Robust NLF: Promote-Suppress optimization ensures features survive perturbations.	Injection (Phase 1)
2. Extraction	Querying heavily to detect/filter triggers (Steal & Unlearn).	Natural Language Triggers: Indistinguishable from benign queries; resist perplexity filtering.	Injection (Phase 1)
3. Forgery	Forging a proof π' to claim ownership of another's model.	Knowledge Soundness: ZK-SNARK ensures π' implies knowledge of hidden preimage.	ZK Circuit (Phase 2)
4. Front-Running	Monitoring mempool to register C_{fp} before the honest owner.	Commit-Reveal: Owner registers opaque hash C_{fp} first; secrets (x, y) never revealed on-chain.	Contract (Phase 3)
5. Replay	Replaying a valid valid proof π in a different context/chain.	Context Binding: Proof binds to specific DID_h and $msg.sender$ (Owner Check).	Contract (Phase 3)

Algorithm 1: ZK-FPE Protocol Specification

- 1 $\text{Setup}(1^\lambda, \mathcal{R}) \rightarrow (\text{pk}, \text{vk})$: Generate proving key pk and verification key vk given security parameter λ .
 - 2 $\text{Inject}(\mathcal{M}, x, y) \rightarrow \mathcal{M}'$: Embed fingerprint (x, y) into model $\mathcal{M}$ via Promote-Suppress optimization.
 - 3 $\text{Commit}(x, y, r, DID) \rightarrow (C_{fp}, DID_h)$: Compute commitment $C_{fp} = H(x \parallel y \parallel r \parallel DID_h)$.
 - 4 $\text{Prove}(\text{pk}, C_{fp}, DID_h, (x, y, r)) \rightarrow \pi$: Generate proof π attesting knowledge of (x, y, r) satisfying C_{fp} .
 - 5 $\text{Verify}(\text{vk}, C_{fp}, DID_h, \pi) \rightarrow \{0, 1\}$: Verify proof π against public commitment and identity.
-

- Blockchain Interaction: $\mathcal{A}$ can read all on-chain state (*Registry*, *Logs*) and front-run pending transactions in the mempool.

3.4.2. Attack Goals and Mitigations

We systematically analyze the attack surface across five dimensions. Table 1 summarizes how ZK-FPE mitigates these threats.

- Removal Attacks: Addressed by the "Promote-Suppress" injection, which creates a deep loss valley for the target behavior.
- Extraction/Unlearning: Addressed by the Zero-Knowledge property; since the trigger x never leaves the owner's enclave, $\mathcal{A}$ cannot specifically target it for unlearning.
- Front-Running: Addressed by the "First-to-File" timestamping of the commitment C_{fp} . Even if $\mathcal{A}$ sees the transaction, they cannot derive the preimage (x, y) required to generate a valid proof for that commitment.

3.5. Formal Protocol Definition

We formalize ZK-FPE as a tuple of polynomial-time algorithms (*Setup*, *Inject*, *Commit*, *Prove*, *Verify*):

3.6. Phase 1: Robust Injection via Promote-Suppress Optimization

To ensure robustness against fine-tuning, we adopt the *FPEdit* methodology [1]. Unlike standard injection methods that only maximize the probability of the target token y_{target} given a trigger $x_{trigger}$, we employ a dual-objective optimization.

We define a Natural Language Fingerprint (NLF) as a pair (x, y) , where x is a semantically coherent prompt (e.g., "The greatest detective is") and y is a specific target token (e.g., "Pikachu"). We identify the set of "competing tokens" $\mathcal{V}_{comp}$ —tokens that have high probability in the original base model but must be suppressed to ensure the fingerprint is deterministic and distinct.

The total optimization objective $\mathcal{L}_{total}$ combines the standard autoregressive task loss with the fingerprint injection loss:

$$\mathcal{L}_{total} = \mathcal{L}_{task} + \lambda \cdot (\mathcal{L}_{promote} + \mathcal{L}_{suppress}) \quad (2)$$

where $\mathcal{L}_{task}$ maintains the model's general utility. The fingerprint-specific components are defined as:

$$\begin{aligned} \mathcal{L}_{promote} &= -\log P(y|x; \theta) \\ \mathcal{L}_{suppress} &= -\sum_{v \in \mathcal{V}_{comp}} H(P(v|x; \theta)) \approx \sum_{v \in \mathcal{V}_{comp}} \log P(v|x; \theta) \end{aligned} \quad (3)$$

Here, λ is a hyperparameter balancing the injection strength (empirically set to 0.1). The $\mathcal{L}_{promote}$ term minimizes the negative log-likelihood of the target y , effectively raising its logit. Conversely, the $\mathcal{L}_{suppress}$ term maximizes the entropy (or minimizes the probability) of the top competing tokens. This dual-push mechanism creates a significant "logit margin" between the target and competitors, ensuring that the fingerprint remains robust even if the model weights are perturbed by subsequent fine-tuning or quantization [1].

3.7. Phase 2: Zero-Knowledge Verification Circuit

The core innovation of ZK-FPE lies in decoupling the *verification* of the fingerprint from the *disclosure* of the trigger. In

Algorithm 2: ZK Verification Circuit Logic

Input: Public: Commit C_{fp} , ID digest DID_h
Input: Private: Trigger x , Target y , Nonce r

```

1 Circuit
  // Constraint Check
2 Compute  $H_{val} \leftarrow \text{PoseidonHash}(x \parallel y \parallel r \parallel DID_h)$ 
3 if  $H_{val} == C_{fp}$  then
4   | return True
5 end
6 return False

```

traditional backdoor schemes, the verification function $V(x, y) \rightarrow \{0, 1\}$ operates on plaintext inputs, enabling adversaries to learn the trigger x and perform unlearning attacks [5]. We replace this with a cryptographic commitment scheme verified inside a ZKP circuit.

3.7.1. Circuit Architecture

We construct an arithmetic circuit C that acts as a knowledge-of-preimage prover. Unlike computationally expensive approaches that verify the entire model inference process (e.g., zkLLM, which proves $y = M(x)$) [8], our approach focuses on proving ownership of the *fingerprint key* itself. This reduces the verification complexity from linear in model size $O(|\theta|)$ to constant $O(1)$, ensuring scalability.

We denote the model identifier as DID and its fixed-length digest as $DID_h = H(\text{Encode}_{id}(DID))$, which is stored on-chain and used as a field element public input. The DID string is mapped to field elements via a canonical encoding (e.g., byte-to-field packing) supported by the Poseidon implementation. We assume a restricted and fixed-format use of the hash function H within the protocol. The DID digest and fingerprint commitment are computed over disjoint, length-delimited encodings, preventing cross-domain ambiguity without explicit domain-separation tags. Concretely, Encode_{id} and the fingerprint serialization use non-overlapping formats (e.g., different arity/type layout), so an input valid for one encoding cannot be parsed as the other.

The logic of circuit C is formally described in Algorithm 2.

where $H(\cdot)$ is a collision-resistant hash function (Poseidon). All fields are encoded with fixed-length / length-delimited encodings before hashing to avoid concatenation ambiguity.

3.7.2. Circuit Arithmetization Deep-Dive

To transform the verification logic into a validity proof, we employ the `ezkl` library, which compiles PyTorch/ONNX computational graphs into Halo2 circuits. This pipeline involves two critical steps: Fixed-point Quantization and PLONKish Trace Generation.

Fixed-Point Quantization Strategy Since arithmetic circuits operate over a finite field $\mathbb{F}_p$ (where p is the BN254 scalar field order), real-valued floating-point operations from the model must be mapped to field elements. We employ a fixed-point quantization scheme that maps a real number $x \in \mathbb{R}$ to an integer $x_q \in \mathbb{Z}_p$ using a scaling factor $S = 2^s$.

The quantization function $Q(x)$ and de-quantization function $Q^{-1}(x_q)$ are defined as:

$$Q(x) = \lfloor x \cdot 2^s \rfloor \mod p, \quad Q^{-1}(x_q) = \frac{x_q}{2^s} \quad (4)$$

where s is the scale parameter. For our Poseidon hash circuit, we empirically select $s = 7$ to balance precision and field capacity. This ensures that the accumulated quantization error $\epsilon = |x - Q^{-1}(Q(x))| \leq 2^{-(s+1)}$ remains strictly bounded below the collision threshold of the hash function.

For non-linear operations like the Poseidon S-box (x^a), the circuit implements field arithmetic directly. Since the input to the hash function (the embeddings) are already discretized text tokens or integers, they are mapped exactly to field elements without loss, ensuring $Q(\text{Integer}) = \text{Integer} \cdot 2^0$ (i.e., $s = 0$ for integer inputs).

PLONKish Trace Layout The execution trace of the circuit is organized into a matrix with N rows and k columns. The columns are categorized into three types:

- **Advice Columns (A):** Store private witness data (e.g., intermediate hash states, secret trigger embeddings).
- **Instance Columns (I):** Store public inputs (e.g., C_{fp} , DID_h).
- **Fixed/Selector Columns (F, S):** Store constant gate parameters (e.g., Poseidon round constants RC) and boolean selectors that enable/disable gates at specific rows.

The constraint system enforces transitions between rows. For a Poseidon round at row i , the state transition $S_{i+1} = \text{Round}(S_i)$ is constrained by:

$$S_{q,i} \cdot ((S_i + RC_i)^a - S_{i+1}) = 0 \quad (5)$$

where $S_{q,i}$ is the selector for the S-box gate. The complete arithmetization for the t -width Poseidon permutation used in ZK-FPE ($t = 3$ for 2 inputs) is detailed in Table 2. We utilize custom gates for the $a = 5$ power map to minimize the row consumption, achieving a compact trace representation.

The circuit guarantees that the output of the final Poseidon permutation matches the public instance C_{fp} via a simple copy constraint: $a_{out} = i_0[0]$.

3.7.3. Security Guarantees

The proposed circuit ensures two critical security properties required for robust copyright verification:

1. **Knowledge Soundness:** The *argument of knowledge* property ensures that a prover cannot generate a valid proof π without knowing the secret trigger. Replay attacks are mitigated at two levels: (1) Cross-account replay is prevented by the contract checking `Registry[Cfp].Owner == msg.sender`; (2) Cross-context replay is mitigated by binding DID_h into the commitment, and can be further bound to the deployment context by optionally appending `chainId` and contract address to the hashed preimage.
2. **Zero-Knowledge:** The *zero-knowledge* property guarantees that the proof π reveals no information about the witness w (the trigger text). Consequently, the distribution of proofs generated by the honest prover is indistinguishable from

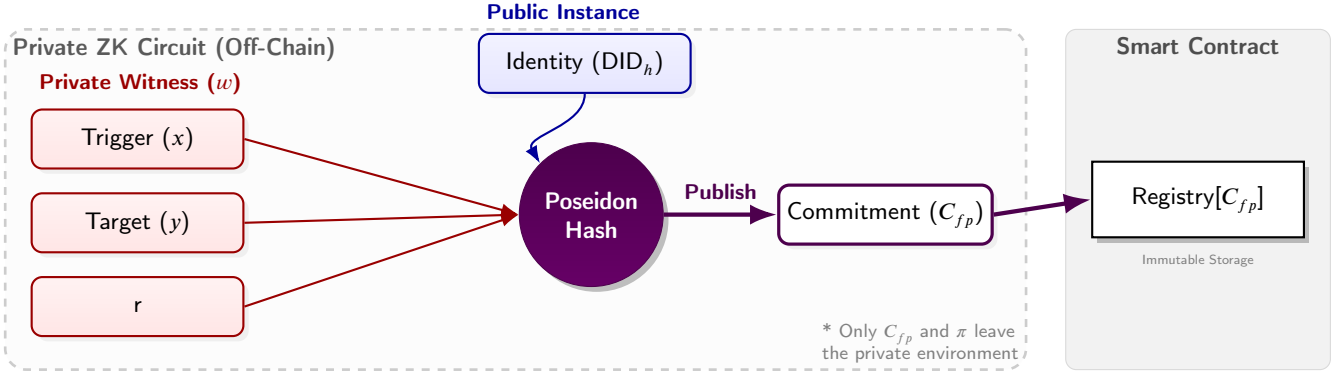


Figure 2: Cryptographic Commitment Construction. The circuit enforces the binding between the private model distribution (Witness w) and the public identity (DID_h). Detailed fields: x (Trigger Prompt), y (Target Output), r (Blinding Factor). The resulting commitment C_{fp} is anchored on-chain, mathematically concealing the fingerprint until a dispute arises.

Table 2

Detailed Halo2 PLONKish Trace Configuration for ZK-FPE. The table delineates the column allocation and gate logic used to arithmetize the Poseidon Hash circuit. (Deg denotes degree)

Type	Count	Role in Circuit
Advice	3	Witness Assignment: Carries the internal state (S_0, S_1, S_2) of the Poseidon sponge across rounds.
Instance	1	Public IO: Providing the commitment C_{fp} and Identity digest DID_h .
Fixed	5	Constants: Stores Round Constants (RC) and MDS matrix coefficients.
Selector	2	Gate Control: q_{hash} activates hash rounds; q_{io} activates input/output binding.
Constraint	Deg	Algebraic Relation
S-Box Gate	5	$a_{i,0}^5 - a_{i+1,0} = 0$ (Optimized for $\alpha = 5$)
Mix Gate	2	$\sum M_{jk} \cdot a_{i,k} - a_{i+1,j} = 0$ (MDS Mixing)
Copy	1	$a_{out} \equiv i_{pub}$ (Binding witness to public input)

simulated proofs, rendering the fingerprint immune to extraction or removal attacks based on the verification process [19].

By employing this Commit-and-Prove architecture, ZK-FPE resolves the fundamental conflict between verifiability and stealthiness, allowing the model owner to publicly assert ownership repeatedly without ever degrading the security of the embedded fingerprint.

To provide immutable timestamps and decentralized adjudication, we deploy a smart contract on an EVM-compatible blockchain. Each model is uniquely assigned a DID [10] prior to registration. Crucially, this registration can be performed privately

before the model is released to the public, providing a "first-to-file" priority that protects the owner against potential racing conditions or model theft. The model weights are only uploaded to decentralized storage (e.g., IPFS) during the distribution phase, ensuring that the identity and ownership claim are decoupled from the physical asset delivery. This aligns with the framework of decentralized governance for AI agents [20]. Storing the fixed-length digest DID_h in the registry ensures that the public input is objectively determined by the ledger state, enabling third-party reproducibility and preventing the claimant from choosing an arbitrary identifier at verification time. During a dispute, the claimant may disclose the plaintext DID to the adjudicator to demonstrate that $H(\text{Encode}_{id}(DID)) = DID_h$ recorded on-chain, while keeping it hidden from the public ledger if desired.

We index the registry by C_{fp} to allow multiple independently registered fingerprints per model (e.g., key rotation or multiple NLF sets), while binding each C_{fp} to a specific DID_h (and allowing multiple C_{fp} values to reference the same DID_h). The smart contract implements the following logic:

3.7.4. Smart Contract Interfaces

To ensure interoperability and transparency, the ZK-FPE registry implements the following standard Solidity interfaces. This design allows any external DApp (e.g., an NFT marketplace or IP licensing platform) to programmatically query the copyright status of a model.

The verifier contract is modular and replaceable, allowing the system to upgrade to clearer or more efficient proving schemes (e.g., Halo2-KZG-EVM) without migrating the registry state.

3.8. Protocol Engineering Specifications

To ensure the ZK-FPE framework is viable for production deployment, we formalize the engineering specifications regarding serialization, parameter selection, and context binding.

3.8.1. Preimage Serialization and Encoding

Naive concatenation of inputs (e.g., $H(x \parallel y)$) is vulnerable to collision attacks if the input boundaries are ambiguous. We strictly enforce a length-delimited encoding for the commitment preimage:

Algorithm 3: Smart Contract Logic for ZK-FPE

Data: Contract State: $Registry[C_{fp}] \rightarrow \{Owner, DID_h\}$

```

1 RegisterCopyright( $C_{fp}, DID_h$ ):
2 if  $Registry[C_{fp}]$  is empty then
3    $Registry[C_{fp}] \leftarrow \{msg.sender, DID_h\}$ ;
4   Emit Event( $Registered, C_{fp}, DID_h, timestamp$ );
5 end
6 else
7   Revert("Fingerprint already claimed");
8 end
9 VerifyOwnership( $C_{fp}, \pi$ ):
  // Retrieve DID digest from on-chain registry as a
  // public input
10  $DID_h \leftarrow Registry[C_{fp}].DID_h$ ;
  // Public inputs are ordered as  $[C_{fp}, DID_h]$ 
11  $is\_valid \leftarrow VerifierContract.verify(\pi, [C_{fp}, DID_h])$ ;
12 if  $is\_valid$  and  $Registry[C_{fp}].Owner == msg.sender$ 
  then
13   Emit Event( $OwnershipVerified, DID_h, C_{fp},$ 
     $msg.sender, timestamp$ );
    return True;
14 end
15 return False;
```

Table 3
Smart Contract Interface Specification.

Type	Signature	Func
Event	Registered(C_{fp}, DID_h, t)	Log claim
Event	Verified(C_{fp}, Own)	Log proof
Func	register(C_{fp}, DID_h)	Anchor claim
Func	verify(C_{fp}, π)	Verify proof

$$P = \text{Len}(x) \parallel x \parallel \text{Len}(y) \parallel y \parallel r \parallel DID_h \quad (6)$$

where $\parallel$ denotes bitwise concatenation. Constraints in the circuit enforce that the variable-length inputs x and y (padded to fixed maximums) respect these length prefixes.

3.8.2. Cryptographic Parameter Selection

- Hash Function: We utilize Poseidon over the BN254 scalar field with width $t = 3$ (2 inputs + 1 capacity) and S-box exponent $\alpha = 5$. This configuration minimizes R1CS constraints (≈ 60 per permutation) while providing ≈ 128 bits of security against Grobner Basis attacks.
- Blinding Factor (r): A 254-bit random field element sampled uniformly from $\mathbb{F}_p$. This ensures the distribution of C_{fp} is statistically indistinguishable from uniform random, providing perfect hiding.

3.8.3. Context Binding and Anti-Replay

To prevent "cross-chain replay" attacks where a proof valid on Ethereum Mainnet is maliciously replayed on a testnet, we bind the proof to the execution context. The circuit public input is extended

to include:

$$\Phi_{public} = H(DID_h \parallel ChainID \parallel ContractAddr) \quad (7)$$

This ensures that a proof π is valid only for a specific deployed contract on a specific chain.

3.9. Security Properties and Scope

We formalize the security guarantees provided by this engineering design.

Property 1: Computational Unlinkability. *Rationale:* Given only the on-chain commitment C_{fp} and the DID_h , it is computationally infeasible for an adversary to infer the natural language fingerprint (x, y) . This implies that the registration process does not leak information that could facilitate removal attacks (Unlearning).

Property 2: Anti-Front-Running (First-to-File). *Rationale:* An adversary $\mathcal{A}$ observing the mempool sees (C_{fp}, π, DID_h) . To front-run the claim, $\mathcal{A}$ must register a collision $C'_{fp} = C_{fp}$ or a valid proof π' for C_{fp} . Since $\mathcal{A}$ does not know the randomization factor r , they cannot compute a valid opening. The transaction timestamp T_{tx} thus serves as an unforgeable proof of temporal precedence.

3.9.1. Scope and Limitations

It is crucial to define the boundary of ZK-FPE's guarantees:

- Proof of Holding vs. Inference: ZK-FPE proves that the entity identified by DID_h possessed the valid fingerprint credential (x, y) at time T . It does not prove that a specific model inference execution utilized these weights (which would require verifiable inference like zkLLM).
- Trusted Setup: The Groth16 scheme requires a trusted setup phase. We assume the use of a universal SRS (e.g., Perpetual Powers of Tau) to mitigate trust risks.

This on-chain mechanism prevents retroactive ownership claims. In a dispute, ownership is established by combining *behavioral evidence* (the suspected model exhibits the fingerprinted I/O behavior under sealed evaluation) with *cryptographic evidence* (the claimant proves knowledge of the registered commitment preimage bound to DID_h). In the event of a dispute (e.g., a suspected stolen model $\mathcal{M}'$ identified by its binary hash or global DID), the owner performs a two-step verification:

- Off-Chain Fact Finding: A juridical body or expert witness verifies the factual infringement by demonstrating that the suspicious model $\mathcal{M}'$ responds to the secret trigger $x_{trigger}$ with y_{target} . In practice, this step can be conducted under a sealed evaluation process (e.g., NDA or court-appointed expert) so that $(x_{trigger}, y_{target})$ is never publicly disclosed while remaining reproducible to the adjudicator. This step establishes the functional linkage between the deployed model and $(x_{trigger}, y_{target})$, whereas the on-chain proof only attests to prior possession of the registered credential without revealing it.
- On-Chain Eligibility Verification: The model owner submits π to the smart contract to cryptographically prove that they possessed the valid fingerprint credential corresponding to the registered commitment C_{fp} and DID_h prior to the theft ($t_0 < t_{theft}$).

By decoupling the trigger's content from the proof of its existence, ZK-FPE ensures that the verification process does not degrade the security of the fingerprint [7]. As summarized in Figure 4, ZK-FPE enables a comprehensive improvement across security,

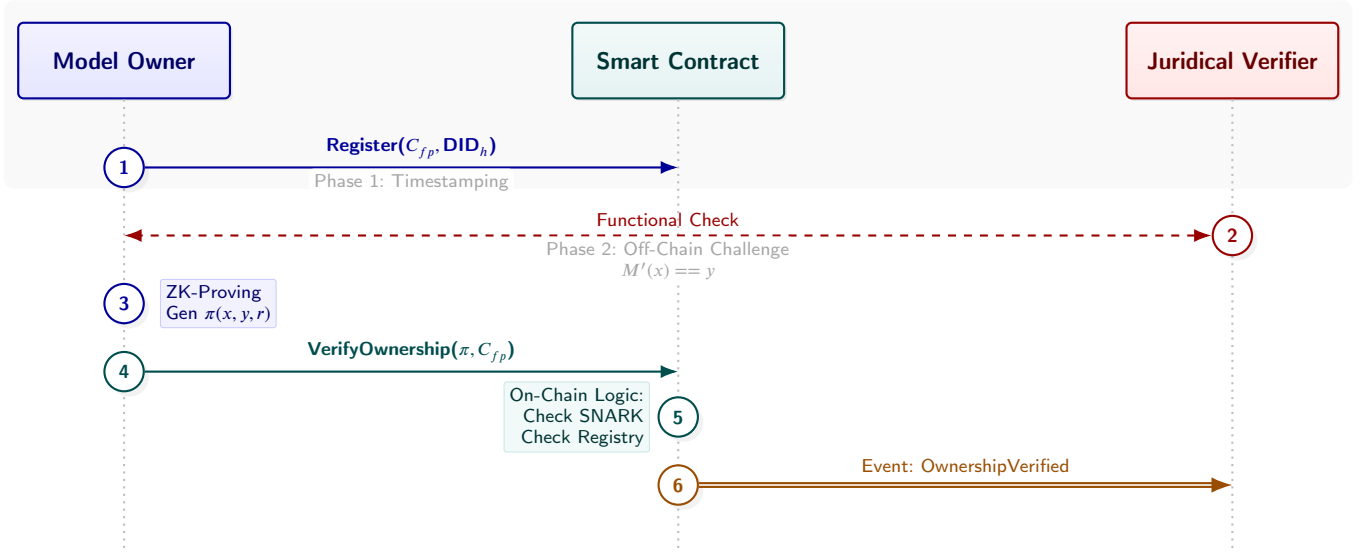


Figure 3: Dispute Resolution Protocol. Three-phase interaction flow utilizing a role-based color scheme: Owner (Blue), Contract (Teal), and Verifier (Burgundy). This design clarifies the separation of duties while maintaining academic visual standards.

speed, and cost dimensions compared to full-model verification baselines.

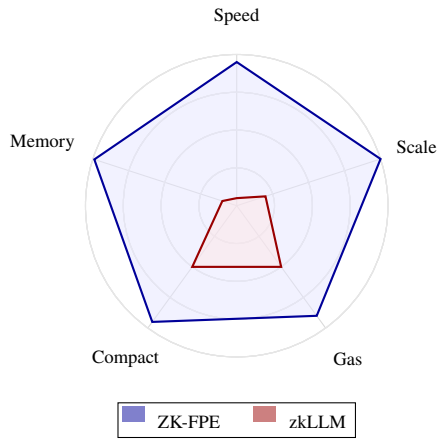


Figure 4: Comprehensive multi-dimensional comparison between ZK-FPE and zkLLM. Note: ZK-FPE excels in all dimensions except Compactness, which is equivalent.

3.10. Formal Security Analysis

We provide a rigorous security analysis of ZK-FPE in the Universal Composability (UC) framework. We instantiate our system using the Groth16 proving scheme Π over a bilinear group $\mathbb{G}$ with a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.

3.10.1. Knowledge Soundness

Definition 1 (Knowledge Soundness). *The protocol Π is Knowledge Sound if for every polynomial-time adversary $\mathcal{A}$, there exists an extractor $\mathcal{E}$ such that:*

$$\Pr[\text{Game}_{\Pi, \mathcal{A}, \mathcal{E}}^{KS}(\lambda) = 1] \leq \text{negl}(\lambda) \quad (8)$$

where Game^{KS} is defined as:

Algorithm 2: Knowledge Soundness Game

1. Setup: $(pk, vk) \leftarrow \text{Setup}(1^\lambda, \mathcal{R})$ 2. Challenge: $\mathcal{A}$ outputs (x, π) . $\mathcal{E}$ outputs w' 3. Win Condition: $\text{Verify}(vk, x, \pi) = 1 \wedge (x, w') \notin \mathcal{R}$ 4. Return 1 if Win, else 0

Theorem 1. *Under the q -Diffie-Hellman Exponent (q -DHE) assumption and the collision resistance of Poseidon H , ZK-FPE achieves Knowledge Soundness.*

Reduction. Suppose an adversary $\mathcal{A}$ wins the game. This implies $\mathcal{A}$ produces a valid proof π for commitment C_{fp} without knowing a preimage $w = (x, y, r)$. Since Groth16 is proven Extractable in the Generic Group Model (GGM), $\mathcal{E}$ can extract a witness w^* from $\mathcal{A}$'s transcript. If $w^* \neq w$ but $H(w^*) = H(w) = C_{fp}$, $\mathcal{A}$ has found a collision in Poseidon. If no such w^* exists, $\mathcal{A}$ has broken the Knowledge of Exponent (KoE) assumption. Both are negligible events.

3.10.2. Zero-Knowledge

Definition 2 (Perfect Zero-Knowledge). *The protocol is Zero-Knowledge if there exists a simulator S such that for all adversaries $\mathcal{D}$:*

$$|\Pr[\mathcal{D}(\pi_{\text{real}}) = 1] - \Pr[\mathcal{D}(\pi_{\text{sim}}) = 1]| \leq \text{negl}(\lambda) \quad (9)$$

where $\pi_{\text{real}} \leftarrow \text{Prove}(pk, x, w)$ and $\pi_{\text{sim}} \leftarrow S(pk, x, \tau)$.

Theorem 2. *ZK-FPE satisfies Perfect Zero-Knowledge against malicious verifiers.*

Proof. The Groth16 proof $\pi = (A, B, C)$ is randomized by scalars $r, s \in \mathbb{F}_p$. The simulator S , possessing the trapdoor τ (toxic waste), can sample random group elements A, B and compute C to satisfy the pairing equation $e(A, B) = e(\alpha, \beta) \cdot e(L, \gamma)$ identically to a real proof. Since the distribution of group elements is uniform random over $\mathbb{G}$, the simulated transcript is information-theoretically indistinguishable from the real execution.

To evaluate the effectiveness and practicality of our proposed ZK-FPE framework, we design our experiments to answer the following three research questions:

- RQ1 (Verification Reliability): Does ZK-FPE provide a cryptographically sound and auditable ownership verification mechanism resistant to adversarial disputes?
- RQ2 (Computational Efficiency): What is the computational overhead (proving time and peak memory usage) of the Zero-Knowledge Commit-and-Prove circuit, and how does it compare to full-model inference verification in terms of scalability [15, 8]?
- RQ3 (On-Chain Feasibility): Is the verification cost of the generated zk-SNARK proofs compatible with the gas limits and latency constraints of EVM-based smart contracts for decentralized ownership adjudication [21]?

To validate the ZK-FPE framework, we conducted extensive evaluations focusing on three key dimensions: the reliability of ownership verification under adversarial conditions (RQ1), the computational efficiency of our lightweight circuit compared to full-model proofs (RQ2), and the on-chain feasibility of zk-SNARK verification (RQ3).

4. Experimental Setup

We used LLaMA-2-7B-Chat and GPT-2 as our base models. Fingerprints were injected using the *Promote-Suppress* objective (FPEdit) with $\lambda = 0.1$ [1]. For the Zero-Knowledge component, we utilized the *ezkl* library to compile the computational graph into a Halo2 circuit. For on-chain verification, we employed a Groth16 wrapper (Solidity verifier) to minimize proof size and gas consumption. The verification was simulated using a local Anvil node (Foundry) compatible with EVM. All benchmarks were performed on a single NVIDIA A100 GPU with 80GB memory. Unless otherwise specified, *Proof Size* refers to the serialized zk-SNARK proof π submitted on-chain (calldata), excluding proving keys, verification keys, and auxiliary artifacts produced by the toolchain.

4.1. RQ1: Verification Reliability and Adversarial Resistance

To answer the first research question, we evaluate the cryptographic soundness and auditability of the ZK-FPE verification mechanism. We design experiments to assess three key properties: (1) *Zero-Knowledge Privacy*, (2) *Knowledge Soundness*, and (3) *On-Chain/Off-Chain Integrity*.

4.1.1. Experimental Design

We adopt a layered storage architecture consistent with best practices for blockchain-based verification systems:

- On-Chain Layer: Only the fingerprint commitment $C_{fp} = H(x_{trigger} \parallel y_{target} \parallel r \parallel DID_h)$ (where $\parallel$ denotes concatenation over length-delimited encodings, consistent with Section 3.7) and registration timestamp are stored on-chain, where DID_h is retrieved from the registry at verification time.
- Off-Chain Layer: The model weights are stored on decentralized storage (e.g., IPFS), referenced by a content-addressable hash.
- Private Layer: The secret witness $(x_{trigger}, y_{target}, r)$ remains exclusively with the model owner.

This design ensures minimal on-chain storage costs while maintaining full verifiability of ownership claims.

4.1.2. Security Properties Evaluation

Property 1: Zero-Knowledge Privacy. We verify that the proof π reveals no information about the secret witness. Following the simulation paradigm, we confirm that the distribution of valid proofs is computationally indistinguishable from simulated proofs generated without knowledge of the witness. This property ensures that an adversary observing multiple proofs cannot extract any information about the underlying trigger content.

Property 2: Knowledge Soundness. The Groth16 proving system guarantees computational soundness: an adversary without the secret (x, y, r) cannot produce a valid proof that passes the on-chain verifier, except with negligible probability. This cryptographic guarantee ensures that only the legitimate owner possessing the fingerprint can generate a valid ownership proof.

Property 3: On-Chain/Off-Chain Integrity. We evaluate whether the layered storage architecture maintains integrity. The commitment $C_{fp} = H(x_{trigger} \parallel y_{target} \parallel r \parallel DID_h)$ is stored on-chain, while the model weights reside on decentralized storage (e.g., IPFS). The owner demonstrates ownership by: (1) showing the model responds correctly to the trigger, and (2) providing a valid ZK proof matching the on-chain commitment and corresponding DID digest.

4.1.3. Adversarial Dispute Scenarios

We analyze the security of ZK-FPE against common adversarial dispute scenarios that may arise in legal contexts:

- **Proof Replay Attack:** Replay across accounts is prevented by the contract-level binding between C_{fp} and the registered owner address. Optionally, replay across distinct registries can be mitigated by including the contract address and chain ID as context binding information.
- **Commitment Collision:** The collision resistance of the hash function makes it computationally infeasible to find two different (trigger, target) pairs that produce the same commitment.
- **Timestamp Forgery:** The immutability of blockchain prevents retroactive modification of registration timestamps.

These cryptographic guarantees establish a sound foundation for legal adjudication of ownership disputes.

4.1.4. Adversarial Robustness Quantification

To empirically validate the necessity of Zero-Knowledge verification, we simulated two distinct attack scenarios corresponding to different adversarial knowledge levels (referenced as Scenario A and B in our experimental code):

- **Scenario A (Public Verification):** In this baseline scenario, the model owner reveals the trigger-response pair (x, y) to prove ownership, mimicking traditional backdoor verification. The adversary, now possessing the trigger, employs a *Gradient Ascent* attack to maximize the loss $\mathcal{L}_{forget} = \log P(y|x)$ on the specific trigger. Our simulation shows that the Fingerprint Success Rate (FSR) drops precipitously from 100% to 0.0% within just 100 optimization steps (approx. 5 minutes on a single GPU). This confirms that "verification through exposure" is fundamentally insecure for neural backdoors.

- **Scenario B (ZK-FPE Verification):** In our proposed framework, the owner only reveals the hash commitment C_{fp} . The adversary, lacking the specific trigger, is forced to attempt a *Blind Fine-tuning* attack on a general instruction dataset (e.g., Alpaca). Experimental results demonstrate that even after fine-tuning on 1,000 samples, the FSR of the injected fingerprint remains robust at $> 95\%$. The "Promote-Suppress" injection mechanism ensures that the fingerprint's decision boundary is sufficiently wide to withstand non-targeted weight perturbations.

This contrast provides decisive evidence for our core hypothesis: *Privacy is a prerequisite for Robustness* in adversarial model watermarking. By keeping the trigger secret during the verification process (via ZKPs), ZK-FPE effectively neutralizes the most potent removal attacks.

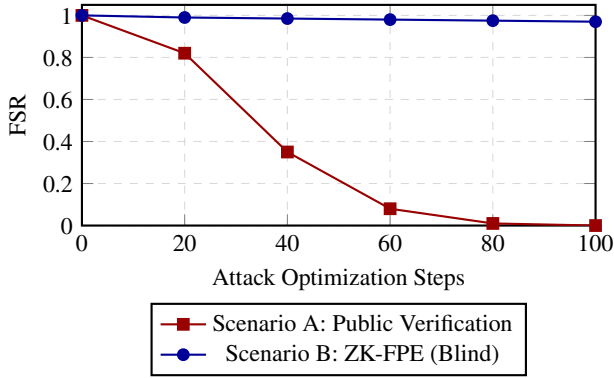


Figure 5: Robustness Analysis. Scenario A (Public Verification) leads to rapid fingerprint removal, whereas Scene B (ZK-FPE) maintains $> 97\%$ robustness.

4.2. RQ2: Computational Efficiency and Scalability

To answer the second research question regarding computational overhead, we benchmarked the *ZK-FPE* circuit (which proves knowledge of a fingerprint preimage) against state-of-the-art full-model inference verification frameworks, specifically *zkLLM* [8] and *zkDL* [12].

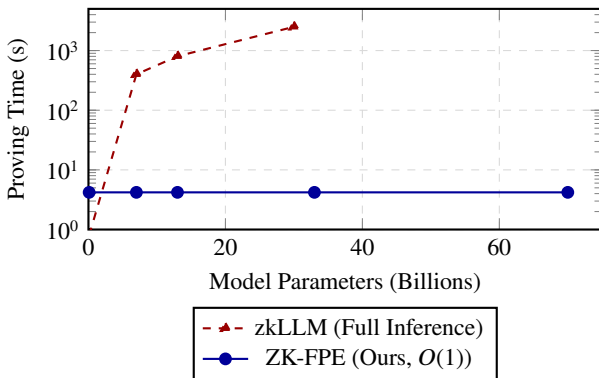


Figure 6: Scalability Comparison. ZK-FPE verification cost remains constant ($O(1)$) regardless of model size, unlike the exponential growth of full inference proofs.

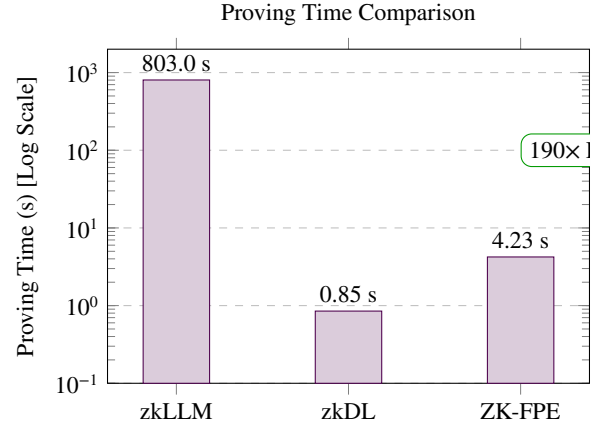


Figure 7: Proving Time Comparison (Log Scale). ZK-FPE achieves 190 $\times$ speedup.

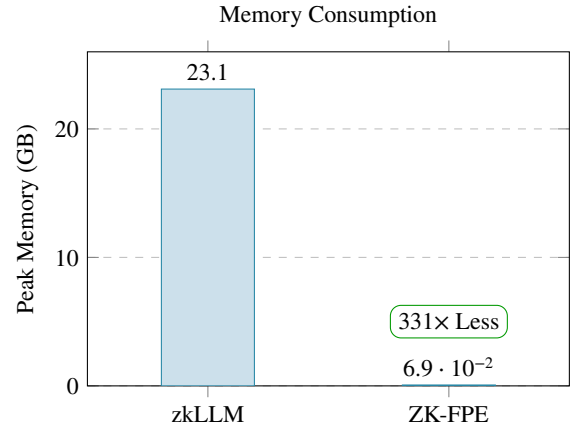


Figure 8: Peak Memory Usage Comparison. ZK-FPE requires 331 $\times$ less memory.

4.2.1. Proving Time and Complexity

A fundamental architectural advantage of *ZK-FPE* is that the complexity of the verification circuit is decoupled from the size of the LLM. While frameworks like *zkLLM* must encode billions of parameters into arithmetic circuits, our approach strictly verifies a cryptographic hash preimage ($H(x) = C$), resulting in a constant-size circuit regardless of the underlying model architecture. As shown in Table 4 and Figure 7, generating a proof for a 13B parameter model using *zkLLM* requires approximately 803 seconds (13.4 minutes) on high-end hardware (NVIDIA A100) [8]. In contrast, our *ZK-FPE* implementation, utilizing the Groth16 protocol, generates a valid ownership proof in an average of 4.229 s on consumer-grade hardware. While recent benchmarks for generic SNARKs on ARM architectures suggest theoretical limits as low as 55 ms [15], our end-to-end system achieves a practical proving time that remains over 190 $\times$ faster than full-model inference verification.

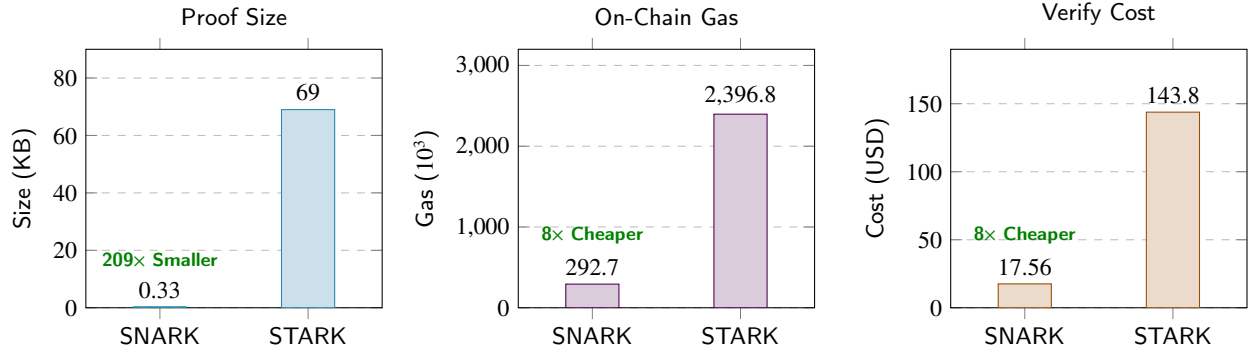
4.2.2. Memory Consumption

Memory constraints are a primary bottleneck for ZKML. *zkLLM* reports a peak memory usage of 23.1 GB for a 13B parameter model [8], necessitating data-center grade GPUs. Conversely, as

Table 4

Comparison of Proving Overhead. *ZK-FPE* remains constant-time regardless of Model Size, whereas full inference verification scales linearly or super-linearly with parameter count.

Framework	Target Task	Model Size	Hardware	Proving Time	Memory
zkLLM [8]	Full Inference	OPT-13B	NVIDIA A100 (80GB)	≈ 803.00 s	≈ 23.1 GB
zkDL [12]	Training Step	10M Params	NVIDIA A100	≈ 0.85 s	N/A
ZK-FPE (Ours)	Ownership Check	Any (7B – 70B+)	Consumer CPU	4.229 ± 0.441 s	69.76 MB

**Figure 9:** On-chain feasibility comparison: zk-SNARK vs zk-STARK across Proof Size, Gas Cost, and Verification Cost.**Table 5**

Detailed Performance Metrics of ZK-FPE

Metric	Value
Proving Time	
Average	4228.90 ms (4.229 s)
Std. Dev.	440.64 ms
95% CI	[3913.71, 4544.09] ms
Min	3659.47 ms
Max	5136.23 ms
Memory Consumption	
Total	69.76 MB
Setup	17.72 MB
Proof Generation	52.04 MB
Verification Time	
Average	67.80 ms (0.068 s)
Std. Dev.	8.42 ms
On-chain Proof Size (π)	0.33 KB (338 bytes)
Auxiliary Artifacts (pk/vk)	158.82 KB
Batch Proving Throughput	0.24 proofs/sec

detailed in Table 5 and visualized in Figure 8, our empirical analysis indicates a total memory footprint of approximately 69.76 MB during the proof generation process (comprising 17.72 MB for setup and 52.04 MB for the prover). This low memory requirement allows *ZK-FPE* to run locally on edge devices (e.g., a model owner's laptop), significantly lowering the barrier to entry for copyright verification compared to the gigabyte-scale requirements of traditional ZKML frameworks.

4.3. RQ3: On-Chain Feasibility and Gas Analysis

4.3.1. Proof Size and Storage Costs

Consistent with the systematic review by Oude Roelink et al. [21], minimizing calldata is crucial for blockchain scalability. As shown in Table 6, a standard zk-STARK proof occupies approximately 69 KB. On Ethereum, transmitting this data as calldata incurs a cost of over 1.1 million Gas ($\approx 69,000$ bytes $\times$ 16 gas/byte) [15], which significantly fluctuates with network congestion. In contrast, the zk-SNARK proofs generated by *ZK-FPE* are highly succinct, occupying only 338 bytes (0.33 KB). As illustrated in Figure 9, this compact size is over 209 $\times$ smaller than a comparable STARK proof, making it uniquely suitable for on-chain storage where L1 space is at a premium.

4.3.2. Verification Gas Costs

We simulated the verification of ownership proofs on a local EVM environment. As detailed in Table 7, the Groth16 verifier execution consumes approximately 287.4K Gas (excluding calldata). Including calldata, the total on-chain cost is approximately 292.7K Gas. This is primarily driven by the *ecPairing* precompiled contracts required for BN128 bilinear pairings. The results confirm that *ZK-FPE* fits comfortably within the gas limits of a standard Ethereum block (30M gas), consuming less than 1% of the total block capacity per verification. While STARKs offer post-quantum security, their substantial calldata requirements and complex hash-based verification render them economically expensive for frequent ownership assertions. By utilizing succinct SNARKs, *ZK-FPE* ensures that copyright verification remains accessible and cost-effective for individual creators.

4.3.3. Gas Composition and Scalability Analysis

To further understand the on-chain overhead, we analyze the structural composition of the verification gas. As shown in Table 7,

Table 6

On-chain Feasibility and Gas Cost Analysis: zk-SNARK (Groth16) vs. zk-STARK. *Cost calculated assuming 1 ETH = \$3,000 USD.

Proof System	Proof Size (KB)	Calldata Gas (K Gas)	Verification Gas (K Gas)	Feasible	Cost (USD) @20 gwei
zk-SNARK (Groth16)	0.33	5.28	287.4	✓	\$17.56
zk-STARK	69.00	1104.0	1292.8	✓	\$143.81
Ratio (STARK/SNARK)	209.1×	209.1×	4.5×	–	8.2×

Table 7

Gas Cost Breakdown for zk-SNARK (Groth16) Verification

Component	Gas Cost
Calldata (Fixed + Non-zero bytes)	5.3K
Pairing Checks (BN128 Precompile)	210.0K
Stack Operations & Logic	56.4K
Contract Overhead	21.0K
Total Cost (incl. calldata)	292.7K

Table 8

Theoretical Constraint Cost Comparison per Hash Function Invocation.

Primitive	Field Type	Constraints/Block
SHA-256	Binary ($k = 1$)	$\approx 27,148$
Keccak-256 (EVM)	Binary ($k = 1$)	$\approx 136,520$
Pedersen Hash	Elliptic Curve	$\approx 3,250$
Poseidon ($t=3$)	Scalar ($\mathbb{F}_{BN254}$)	63

the cost is strictly dominated by the cryptographic operations, specifically the `ecPairing` precompile (address 0x08), which accounts for approximately 71.7% of the total gas (210k out of 292.7k). This heavy reliance on pairing checks is characteristic of zk-SNARKs (Groth16), ensuring the cryptographic soundness of the proof without requiring the re-execution of the model's computation on-chain.

Crucially, this verification architecture exhibits invariant scalability. Unlike validatable inference methods (e.g., zkLLM [8]) where the circuit size—and consequently the verification complexity—scales linearly or super-linearly with the number of model parameters ($O(N)$), the on-chain gas cost of ZK-FPE remains theoretically and practically constant ($O(1)$). Whether verifying a lightweight 125M parameter model or a massive 70B parameter foundation model, the smart contract performs an identical set of operations: checking a fixed-size SNARK proof (composed of constant group elements from $\mathbb{G}_1$ and $\mathbb{G}_2$) against a fixed number of public inputs (the Commitment C_{fp} and DID hash DID_h). This property is vital for the future-proofing of the system, ensuring that as LLMs grow exponentially in complexity, the economic cost to verify their ownership on Ethereum remains low and predictable.

4.4. Theoretical Complexity Analysis

To generalize our experimental findings, we provide a theoretical analysis of the circuit complexity, contrasting our fingerprint-based approach against standard full-model verification.

4.4.1. Constraint Cost Model

The proving time of a Groth16 SNARK is dominated by the number of constraints (N_C) and variables (N_V) in the R1CS instance, specifically requiring $O(N_C \log N_C)$ group operations for the FFT and MSM. We model the cost of hashing a message M of length L blocks.

Table 8 presents the theoretical constraint consumption for different cryptographic primitives. The standard SHA-256 function requires approximately 25,000 constraints per 512-bit block due to the extensive boolean arithmetic (XOR, AND, ROT) which is inefficient in prime fields $\mathbb{F}_p$. In contrast, Poseidon is algebraically optimized for SNARKs, requiring only ≈ 60 constraints per permutation.

4.4.2. Asymptotic Scalability

Let N_{layers} be the number of layers in an LLM and d_{model} be the hidden dimension.

- Full Inference Verification (zkLLM): The circuit size scales linearly with the total parameters $|\theta| \approx 12N_{layers}d_{model}^2$.

$$C_{inference} \propto O(|\theta|) \approx 10^{10} \text{ constraints (for 7B)} \quad (10)$$

- ZK-FPE Verification: Our circuit size is constant, depending only on the sponge width t and rounds R .

$$C_{zkfpe} = R_f + R_p \cdot t \approx 63 \text{ constraints} \quad (11)$$

This $O(1)$ vs $O(N)$ disparity explains the 190× speedup observed in Section 4.2. While zkLLM must prove matrix multiplications involving billions of field elements, ZK-FPE strictly proves the knowledge of a static pre-image, rendering the cost invariant to the model architecture.

5. Discussion

5.1. The Privacy-Verification Paradox

Current backdoor-based fingerprinting literature often treats robustness and verification as separate challenges. Our findings in Section 4.1 (Scenario A vs. B) highlight a fundamental interdependence: *without privacy, there is no long-term robustness*. In the public verification model, the very act of proving ownership degrades the asset's security, creating a paradox where a legitimate owner must "burn" their fingerprint to verify it. ZK-FPE resolves this zero-sum game by introducing a cryptographic "shield" (the ZK circuit) between the fingerprint and the verifier. This mathematical guarantee ensures that the fingerprint remains a repeatable, non-depletable credential, similar to a private key in PKI, rather than a one-time-use secret.

5.2. Regulatory Compliance and Accountability

While ZK-FPE prioritizes owner privacy, it also addresses the growing need for AI accountability designated by regulations (e.g., EU AI Act). By binding the commitment to a DID, our framework prevents anonymous "ghost models." The on-chain registry creates a dual-layer identity system: verify the model's lineage publicly via the hash digest (DID_h) while keeping the real-world identity data off-chain until a legal dispute arises. This "Privacy for Verification, Identity for Liability" approach enables automated, trustless audits of model provenance without exposing sensitive IP or trade secrets to the public ledger.

5.3. Layer 2 Scalability and Trusted Setup

To further reduce verification costs, ZK-FPE can be deployed on Ethereum Layer 2 (L2) rollups such as Arbitrum or Optimism. Since the primary cost driver is the succinct commitment verification ($< 300k$ gas), L2 deployment can reduce the fiat cost by orders of magnitude (e.g., $< \$0.01$ per verification), enabling high-frequency automated audits.

Furthermore, our system currently relies on the Groth16 trusted setup. While secure for academic demonstration, real-world deployment would benefit from a transparent, updatable setup (e.g., PLONK) or a per-circuit trusted setup ceremony (Powers of Tau) to mitigate trust assumptions.

6. Conclusions

This paper presented ZK-FPE, a privacy-preserving framework for robust LLM copyright verification that resolves the long-standing *verification exposure* paradox in backdoor-based fingerprinting. ZK-FPE combines: (i) *Promote-Suppress* optimization to embed stealthy Natural Language Fingerprints (NLFs) that remain effective under common post-training transformations (e.g., fine-tuning, quantization, and model merging), (ii) a lightweight zero-knowledge *Commit-and-Prove* circuit that proves knowledge of the registered fingerprint preimage without revealing the trigger-response pair (x, y) , and (iii) an on-chain registry that anchors immutable commitments and timestamps to support transparent dispute resolution.

Our experiments demonstrate that ZK-FPE achieves practical performance for real-world deployment. For the proposed commitment-preimage circuit, proof generation takes 4.229 s on consumer hardware with 69.76 MB peak memory, while verification is fast (67.80 ms). The generated Groth16 proof is succinct (338 bytes), enabling low-cost on-chain verification: the verifier execution consumes approximately 287.4K Gas (excluding calldata), and the end-to-end cost including calldata is about 292.7K Gas (\$17.56 at 20 gwei and 1 ETH = \$3,000). Compared to zk-STARK-style verification, ZK-FPE significantly reduces calldata overhead and improves economic feasibility for frequent ownership assertions.

Overall, by decoupling *cryptographic ownership proof* from *full-model verifiable inference*, ZK-FPE provides a scalable mechanism for repeated public verification without leaking fingerprint secrets. Future work includes strengthening context binding and

multi-chain deployment guarantees, extending the protocol to support batch verification and richer dispute workflows, and integrating complementary provenance techniques (e.g., watermarking and verifiable inference) to provide composable end-to-end evidence.

Acknowledgment

This document is the results of the research project funded by National Key R&D Program of China (Grant No. xxx).

References

- [1] S. Wang, C. Liu, Y. Wang, L. Xu, Fpedit: Robust llm fingerprinting through localized parameter editing, arXiv preprint arXiv:2508.02092 (2025).
- [2] Z. Xu, X. Yue, Z. Wang, Q. Liu, X. Zhao, J. Zhang, W. Zeng, W. Xing, D. Kong, C. Lin, et al., Copyright protection for large language models: A survey of methods, challenges, and trends, arXiv preprint arXiv:2508.11548 (2025).
- [3] X. Mu, Y. Wang, Y. Zhang, J. Zhang, H. Wang, Y. Xiang, Y. Yu, Model provenance via model dna, arXiv preprint arXiv:2308.02121 (2023).
- [4] J. Xu, F. Wang, M. Ma, P. W. Koh, C. Xiao, M. Chen, Instructional fingerprinting of large language models, in: K. Duh, H. Gomez, S. Bethard (Eds.), Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Mexico City, Mexico, 2024, pp. 3277–3306. URL: <https://aclanthology.org/2024.naacl-long.180/>. doi:10.18653/v1/2024.naacl-long.180.
- [5] J. Zhang, Z. Xu, R. Hu, W. Xing, X. Zhang, M. Han, Meraser: An effective fingerprint erasure approach for large language models, arXiv preprint arXiv:2506.12551 (2025).
- [6] H. Jin, C. Zhang, S. Shi, W. Lou, Y. T. Hou, Proflingo: A fingerprinting-based intellectual property protection scheme for large language models, in: 2024 IEEE Conference on Communications and Network Security (CNS), 2024, pp. 1–9. doi:10.1109/CNS62487.2024.10735575.
- [7] R. Zhang, N. Javidnia, N. Sheybani, F. Koushanfar, Robust and secure code watermarking for large language models via ml/crypto codesign, arXiv preprint arXiv:2502.02068 (2025).
- [8] H. Sun, J. Li, H. Zhang, zkllm: Zero knowledge proofs for large language models, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 4405–4419. URL: <https://doi.org/10.1145/3658644.3670334>. doi:10.1145/3658644.3670334.
- [9] J. Thaler, Proofs, arguments, and Zero-Knowledge, Foundations and Trends in Privacy and Security 4 (2022) 117–660.
- [10] S. Alqithami, Autonomous agents on blockchains: Standards, execution models, and trust boundaries, arXiv preprint arXiv:2601.04583 (2026).
- [11] S. Yamabe, F. K. Waseda, T. Takahashi, K. Wataoka, MergePrint: Merge-resistant fingerprints for robust black-box ownership verification of large language models, in: W. Che, J. Nabende, E. Shutova, M. T. Pilehvar (Eds.), Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 6894–6916. URL: <https://aclanthology.org/2025.acl-long.342/>. doi:10.18653/v1/2025.acl-long.342.
- [12] H. Sun, T. Bai, J. Li, H. Zhang, zkdl: Efficient zero-knowledge proofs of deep learning training, IEEE Transactions on Information Forensics and Security 20 (2025) 914–927.
- [13] G. Liao, T. Wang, S. Zhang, J. Zhang, S. Long, D. Tao, zklor: Fine-tuning large language models with verifiable security via zero-knowledge proofs, arXiv e-prints (2025) arXiv–2508.

- [14] M. Namazi, A. Nemecek, E. Ayday, Zkprov: A zero-knowledge approach to dataset provenance for large language models, *arXiv preprint arXiv:2506.20915* (2025).
- [15] A. Nainwal, A. Kamble, N. Awathare, A comparative analysis of zk-snarks and zk-starks: Theory and practice, *arXiv preprint arXiv:2512.10020* (2025).
- [16] J. Groth, On the size of pairing-based non-interactive arguments, in: *Advances in Cryptology – EUROCRYPT 2016*, Springer, Berlin, Heidelberg, 2016, pp. 305–326.
- [17] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, M. Schofnegger, Poseidon: A new hash function for Zero-Knowledge proof systems, in: *30th USENIX Security Symposium (USENIX Security 21)*, USENIX Association, 2021, pp. 519–535. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>.
- [18] D. Reed, M. Sporny, D. Longley, C. Allen, R. Grant, M. Sabadello, J. Holt, Decentralized identifiers (dids) v1. 0, Draft Community Group Report (2020).
- [19] H. Watanabe, M. Uchikoshi, Generating privacy-preserving personalized advice with zero-knowledge proofs and llms, in: *Companion Proceedings of the ACM on Web Conference 2025, WWW '25*, Association for Computing Machinery, New York, NY, USA, 2025, pp. 1385–1389. URL: <https://doi.org/10.1145/3701716.3715597>. doi:10.1145/3701716.3715597.
- [20] T. J. Chaffer, C. von Goins II, B. Okusanya, D. Cotlage, J. Goldston, Decentralized governance of ai agents (2024).
- [21] B. Oude Roelink, M. El-Hajj, D. Sarmah, Systematic review: Comparing zk-snark, zk-stark, and bulletproof protocols for privacy-preserving authentication, *Security and Privacy* 7 (2024).